

AWS Hyper-Efficient FinOps: Automated Cloud Waste Remediation Engine

Project Description:

The AWS Hyper-Efficient FinOps Automation Engine is a cloud cost optimization workflow designed to detect and remediate unnecessary expenditure in AWS environments. It uses the AWS Cost & Usage Report (CUR) delivered to S3, AWS Glue for data cataloging, and Athena for SQL-based cost analysis.

A serverless remediation bot (AWS Lambda) is scheduled via EventBridge to automatically detect idle resources and reduce cloud waste.

The original project requirement included building a cost dashboard using Amazon QuickSight. However, QuickSight was not available or enabled in this AWS account/region, and Amazon QuickSuite was shown instead. Due to this limitation, visualization dashboards could not be generated, and Excel-based alternatives were not used because the requirement specifically requested QuickSight.

All other FinOps components were successfully implemented.

Scenario 1: Idle EC2 Detection

CUR data identifies EC2 resources with low `line_item_usage_amount`. The remediation Lambda function is designed to flag such resources for shutdown to reduce cost.

Scenario 2: Multi-Service Cost Breakdown

Athena SQL queries help analyze which AWS services contribute most to the cost (via `product_servicecode` and `line_item_unblended_cost`).

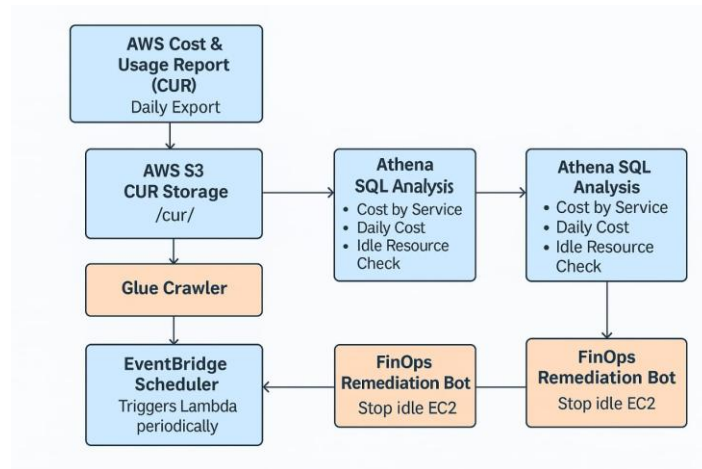
Scenario 3: Automated Remediation

Using EventBridge, the Lambda bot runs on a fixed schedule (e.g., every 6 hours) to identify and react to cost waste.

Scenario 4: CUR Delay

AWS CUR requires 4–24 hours to refresh. An EC2 instance created during testing does not appear immediately but will appear in the next CUR cycle.

Technical Diagram:



ER Diagram:

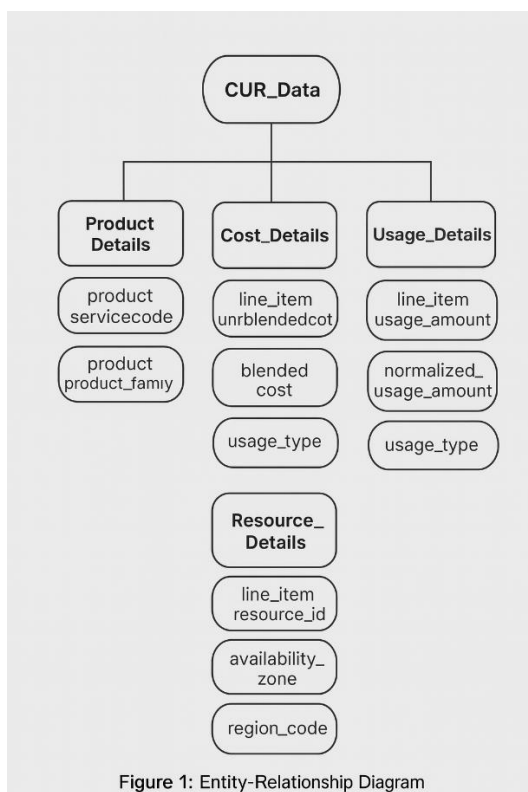


Figure 1: Entity-Relationship Diagram

Prior Knowledge:

1. AWS Account Setup

Understanding how to create an AWS account, enable billing, and navigate console services.

Reference:

https://youtu.be/CjKhQoYeR4Q?si=ui8Bvk_M4FfVM-Dh

2. AWS Cost Management & Cost Explorer

Basics of AWS billing, cost explorer, charge breakdown, and FinOps mindset.

Reference:

<https://youtu.be/OKYJCHHSWb4?si=aY3DQl1v26CfZxXA>

3. AWS S3 (Used for Storing CUR Data)

Creating buckets, working with folders, and handling objects in S3.

Reference:

<https://youtu.be/9vK7fwAhVtA?si=xJHI-RSt3PpFBINK>

4. AWS Cost & Usage Report (CUR)

Understanding how AWS generates CUR, export configuration, formats, and usage for analysis.

Reference:

<https://youtu.be/xtb1adoR1Uc?si=7stvJpMLujIVNvol>

5. AWS Glue (Crawler & Data Catalog)

Using Glue Crawlers to scan S3 data and generate Athena tables.

Reference:

<https://youtu.be/weWeaM5-EHc?si=KdnxrN5TIRwwe8bD>

6. Amazon Athena (SQL Analytics for CUR)

Running SQL queries on CUR data to extract service-wise costs, usage metrics, and idle patterns.

Reference:

https://youtu.be/Wkpl66NaqEA?si=lxioUrZrJ7JrO_9

7. AWS Lambda (Automation for Cost Optimization)

Building serverless functions to automate tasks such as identifying idle EC2 instances.

Reference:

<https://youtu.be/e1tkFsFOBHA?si=X6sel8yfZUfUzir>

8. Amazon EventBridge (Scheduler for Lambda)

Creating scheduled rules (Periodic triggers) for automated remediation.

Reference:

https://youtu.be/TXh5oU_yo9M?si=5YfPKYtvY-3fwaUU

Project Workflow:

1. AWS Account & IAM Setup

- Create an AWS account and configure billing access.
- Enable Billing Preferences → receive cost notifications.
- Create an IAM user with permissions for S3, Glue, Athena, Lambda, and EventBridge.
- Log in to the AWS Management Console using the IAM user credentials.

2. CUR (Cost & Usage Report) Export Setup

- Navigate to the AWS Billing Console → Data Exports.
- Create a Standard CUR 2.0 Export.
- Configure export settings:
 - Include resource IDs
 - Select Parquet format
 - Choose Daily refresh cadence
 - Set S3 destination path: s3://finops-cur-data-ungabunga/cur/
- Enable the export — CUR files begin delivering to S3 (updates every 4–24 hours).

3. S3 Bucket Preparation

- Create a dedicated S3 bucket to store all CUR files.
- Organize folders such as:
 - /cur/ (Cost & Usage Reports)
- Verify newly exported CUR objects appear under the cur/ prefix.

4. AWS Glue Crawler & Data Catalog Setup

- Create a Glue Database named finops_cur_db.
- Create a Glue Crawler linked to your S3 CUR folder.
- Configure crawler:
 - Data source: S3
 - IAM role: GlueCURCrawlerRole
 - Output prefix: cur_
 - Target database: finops_cur_db
- Run the crawler —> it generates a table named cur_data containing CUR fields.
- View schema and validate columns

5. Cost Analysis Using Amazon Athena

- Open Athena and select the database: finops_cur_db.
- Inspect table cur_data (generated by Glue).
- Run analytical queries for cost insights:
 - Cost by service using product_servicecode
 - Daily cost trend using line_item_usage_start_date
 - Idle EC2 usage check using line_item_usage_amount

- Use query results to interpret cloud spending patterns.
- Note: EC2 instance usage appears after next CUR refresh (4–24 hours).

6. Lambda Remediation Bot Setup

- Create a Lambda function named FinOpsRemediationBot.
- Add logic to identify idle EC2 resources based on CUR signals.
- Configure IAM role for Lambda to manage EC2 actions.
- Deploy function and validate that it executes successfully

7. EventBridge Automation Setup

- Create an EventBridge Rule named FinOps-Remediation-Scheduler.
- Set schedule pattern (e.g., every 6 hours) to periodically check for idle resources.
- Attach Lambda function as the target.
- Verify scheduled invocations through EventBridge and CloudWatch logs.

8. Visualization Limitation & Workaround

- Attempted to enable Amazon QuickSight, but the AWS account redirected to Amazon QuickSuite instead (QuickSight unavailable in this region/account).
- QuickSight signup UI did not appear (username/password fields missing).
- Due to this limitation, the dashboard component could not be implemented.
- Cost breakdowns and trends were instead interpreted directly from Athena query results.

9. Monitoring & Maintenance

- Use CloudWatch to monitor:

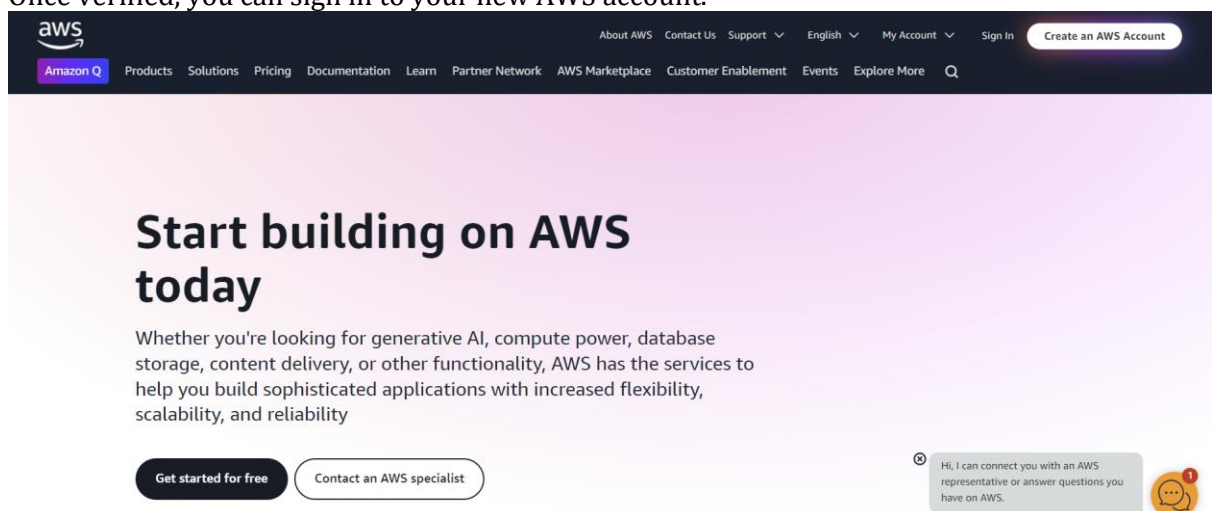
- Lambda function logs
- EventBridge rule triggers
- API throttling or Athena query errors
- Monitor cost updates via Billing Dashboard.
- Validate S3 CUR updates daily (4–24 hour delay normal).
- Review EC2 status changes triggered by remediation bot.

Milestone 1: AWS Account Creation and Login

This milestone establishes the AWS environment needed for FinOps cost analysis and automation.

Activity 1.1: Create AWS Account

1. Go to the AWS website (<https://aws.amazon.com/>).
2. Click on the "Create an AWS Account" button.
3. Follow the prompts to enter your email address and choose a password.
4. Provide the required account information, including your name, address, and phone number.
5. Enter your payment information. (Note: While AWS offers a free tier, a credit card or debit card is required for verification.)
6. Complete the identity verification process.
7. Choose a support plan (the basic plan is free and sufficient for starting).
8. Once verified, you can sign in to your new AWS account.





Sign in

☒ **Root user**

Account owner that performs tasks requiring unrestricted access. [Learn more](#)

☐ **IAM user**

User within an account that performs daily tasks. [Learn more](#)

Root user email address

username@example.com

Next

By continuing, you agree to the [AWS Customer Agreement](#) or other agreement for AWS services, and the [Privacy Notice](#). This site uses essential cookies. See our [Cookie Notice](#) for more information.

— New to AWS? —

Create a new AWS account

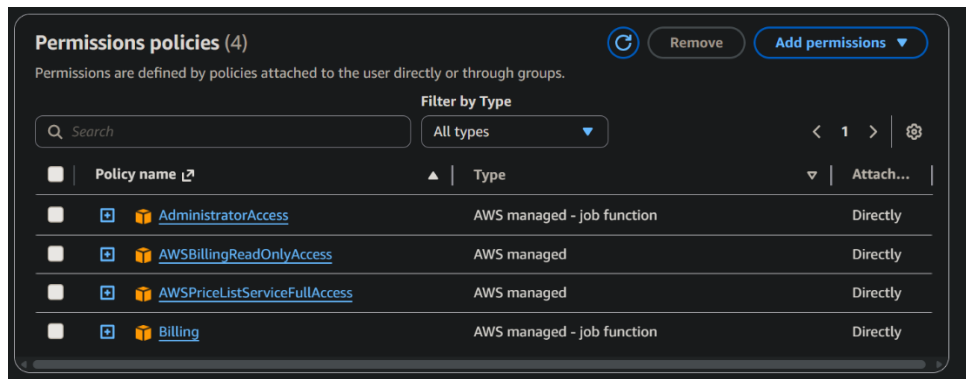


Activity 1.2: Configure Billing Access

1. Open the Billing Dashboard.
2. Enable Billing Alerts and Cost Explorer.
3. Verify access to:
 - Cost & Usage Reports (CUR)
 - Data Exports
 - Budgets
 - Cost Allocation Tools

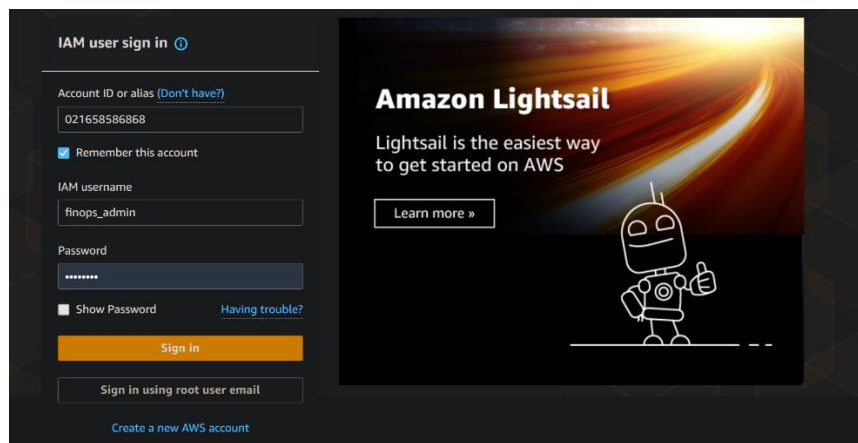
Activity 1.3: Create IAM User

1. Open **IAM Console** → Add User.
2. Enable **Console + Programmatic** access.
3. Attach permissions for S3, Glue, Athena, Lambda, EventBridge, Billing access.
4. Download credentials securely.



Activity 1.4: Login Using IAM User

1. Log out from root account.
2. Login using IAM user for all future operations.

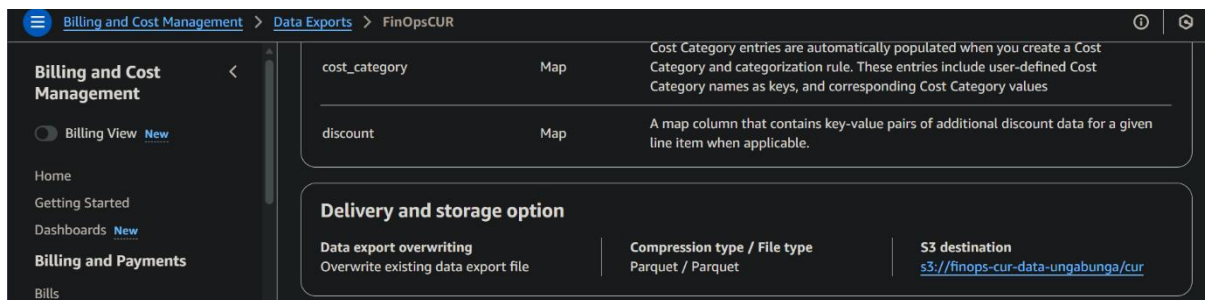
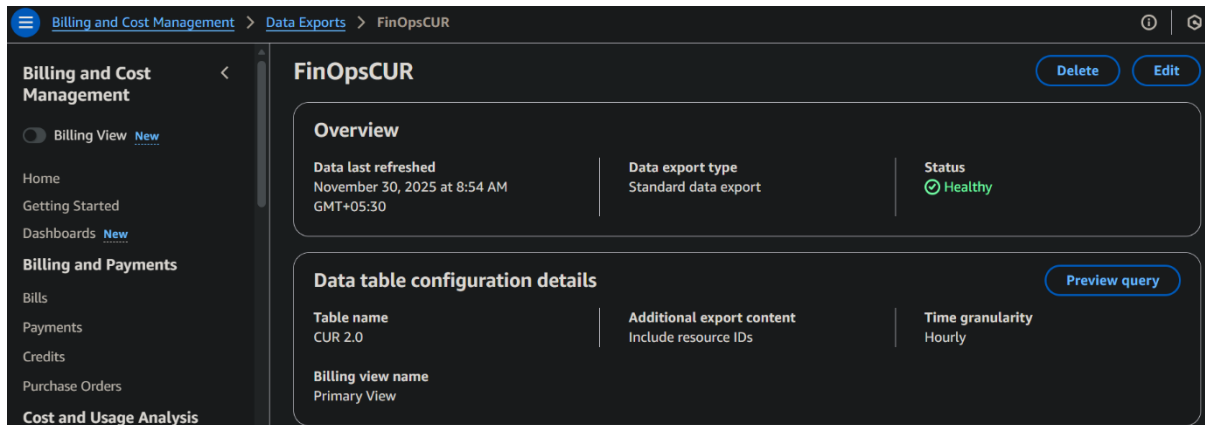
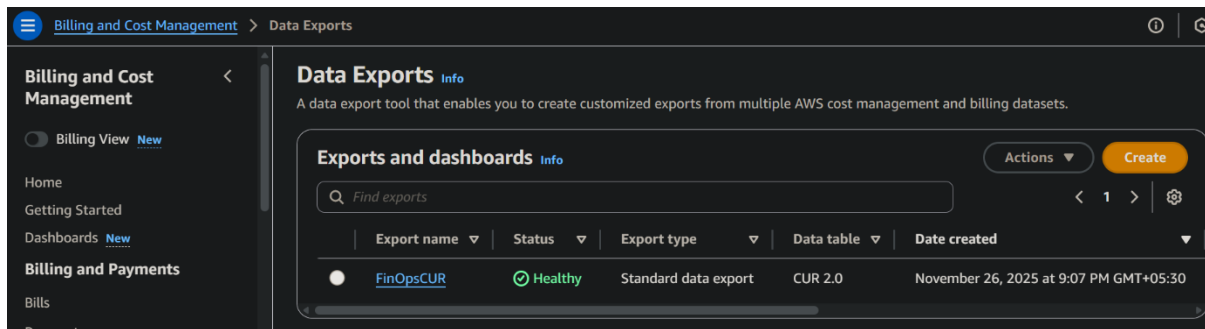


Milestone 2: CUR Export and S3 Setup

This milestone configures the AWS Cost & Usage Report, which forms the foundation for cost analytics.

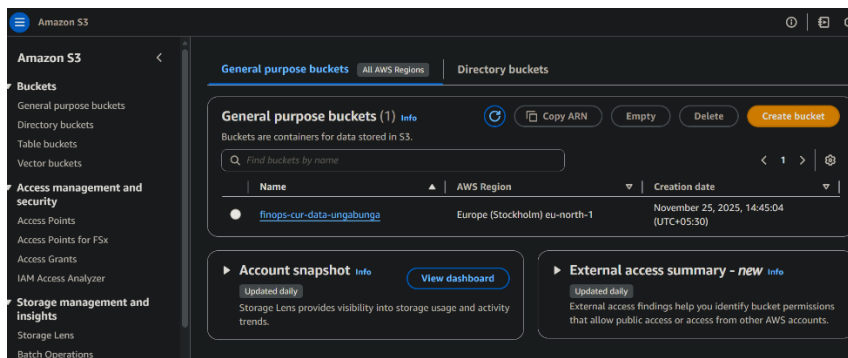
● Activity 2.1: Create Standard CUR Export

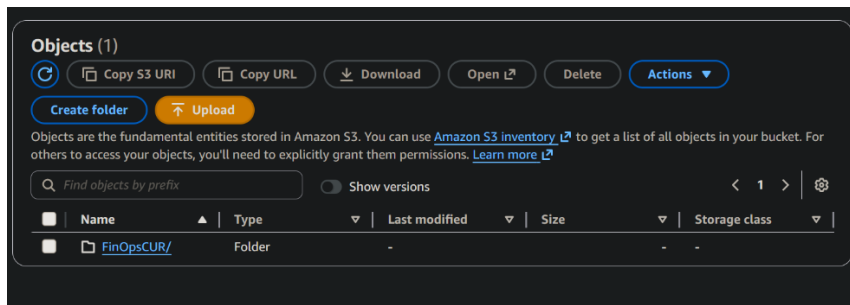
1. Open **Billing** → **Data Exports**.
2. Click **Create Export**.
3. Choose:
 - a. Export Type: Standard
 - b. Data Table: CUR 2.0
 - c. Daily Refresh
 - d. Include Resource IDs
 - e. Parquet Format
4. Set S3 destination:
finops-cur-data-ungabungga/cur/
5. Save and activate the export.



● Activity 2.2: Verify S3 Delivery

1. Navigate to S3 bucket.
2. Open the `/cur/` folder.
3. Confirm that CUR files appear (note: 4–24 hour delay).
4. Observe structure and folder partitions.





● Activity 2.3 Understand CUR Refresh Cycle

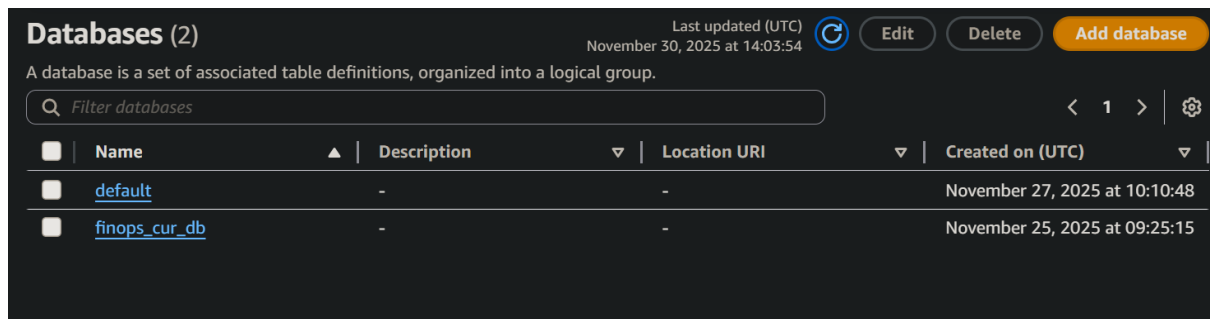
1. CUR updates every **4–24 hours**.
2. New EC2 instance usage will appear only in the next cycle.

Milestone 3: Glue Crawler & Data Catalog Setup

This milestone converts raw CUR data from S3 into a structured table accessible through Athena.

Activity 3.1: Create Glue Database

1. Go to AWS Glue → Databases.
2. Create a database named:
3. `finops_cur_db`



Activity 3.2: Configure Glue Crawler

1. Navigate to Glue → Crawlers.
2. Create crawler named:
3. `finops-cur-crawler`
4. Configure:
 - Source: `S3 /cur/` path
 - Role: `GlueCURCrawlerRole`
 - Output: `finops_cur_db`
 - Prefix: `cur_`
5. Run the crawler.

AWS Glue > Crawlers

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations **New**

Data Catalog

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (1) **Info** Last updated (UTC) November 30, 2025 at 13:58:06 Action Run Create crawler

View and manage all available crawlers.

Name	State	Schedule	Last run	Last run ...	Log	Table cha...
finops-cur-c...	Ready		Succeeded	November 2...	View log	2 updated

AWS Glue > Crawlers > finops-cur-crawler

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations **New**

Data Catalog

- Databases
- Tables

finops-cur-crawler Last updated (UTC) November 30, 2025 at 14:02:52 Run crawler Edit Delete

Crawler properties

Name finops-cur-crawler	IAM role GlueCURCrawlerRole	Database finops_cur_db	State READY
Description -	Security configuration -	Lake Formation configuration -	Table prefix cur_
Maximum table threshold -			

[Advanced settings](#)

Crawler runs | **Schedule** | **Data sources** | **Classifiers** | **Tags**

Crawler runs (3) Stop run View CloudWatch logs View run details

The list of crawler runs for this crawler.

	Start time (UTC)	End time (UTC)	Current/last duration	Status	DPU hours
	November 27, 2025 at ...	November 27, 2025 at ...	46 s	Completed	0.062
	November 27, 2025 at ...	November 27, 2025 at ...	02 min 08 s	Completed	0.039
	November 25, 2025 at ...	November 25, 2025 at ...	44 s	Completed	0.053

Activity 3.3: Validate Table Creation

- 1) Open Glue Database → finops_cur_db.
- 2) Confirm table: cur_data
- 3) Review schema and attributes (line_item, product, pricing fields)

finops_cur_db

Last updated (UTC)
November 30, 2025 at 14:04:00

Edit

Delete

Database properties

Name

finops_cur_db

Description

-

Location

-

Created on (UTC)

November 25, 2025 at 09:25:15

Tables (2)

Last updated (UTC)
November 30, 2025 at 14:04:02

Delete

Add tables using crawler

Add table

View and manage all available tables.

Filter tables

< 1 >

Name

Database

Location

Classific...

Depreca...

View data

Data quality

cur_data

finops_cur_db

s3://finops-cur-c

Parquet

-

Table data

View data quality

cur_metadata

finops_cur_db

s3://finops-cur-c

JSON

-

Table data

View data quality

cur_data

Last updated (UTC)
November 30, 2025 at 14:04:23

Version 2 (Current version)

Actions

Table overview

Data quality - new

Table details

Name

cur_data

Database

finops_cur_db

Description

-

Last updated

November 27, 2025 at 10:07:48

Classification

Parquet

Location

s3://finops-cur-data-ungabunga/cur/FinOpsCUR/data/

Connection

-

Deprecated

-

Column statistics

No statistics

Advanced properties

Schema (115)

Edit schema as JSON

Edit schema

View and manage the table schema.

Filter schemas

< 1 2 3 4 5 6 >

#

Column name

Data type

Partition key

Comment

1

bill_bill_type

string

-

-

2

bill_billing_entity

string

-

-

3

bill_billing_period_e...

timestamp

-

-

4

bill_billing_period_st...

timestamp

-

-

5

bill_invoice_id

string

-

-

6

bill_invoicing_entity

string

-

-

7

bill_payer_account_id

string

-

-

8

bill_payer_account_n...

string

-

-

9

cost_category

map

-

-

10

discount

map

-

-

11

discount_bundled_di...

double

-

-

12

discount_total_disco...

double

-

-

13

identity_line_item_id

string

-

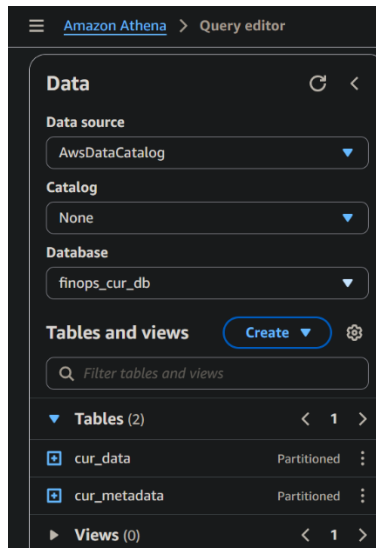
-

Milestone 4: Athena Cost Analysis

In this milestone, we query structured CUR data using Athena to extract billing insights.

Activity 4.1: Connect Athena to Glue Table

1. Open **Athena**.
2. Select Database → finops_cur_db.
3. Select table → cur_data.



Activity 4.2: Run Cost Analytics Queries

Run SQL queries for FinOps insights:

- Cost by Service

Using product_servicecode + line_item_unblended_cost

- Daily Cost Trends

Using line_item_usage_start_date

- Idle Resource Analysis

Using line_item_usage_amount for AmazonEC2

<

Query 1 : ✕

✓ Query 2 : ✕

✓ Query 3 : ✕

✓ Query 4 : ✕

✓ Query 5 : ✕

>

(+) ▼

1 SELECT

2 DATE(line_item_usage_start_date) AS day,

3 SUM(CAST(line_item_unblended_cost AS DOUBLE)) AS cost

4 FROM finops_cur_db.cur_data

5 GROUP BY 1

6 ORDER BY day;

7

SQL Ln 1, Col 1

Run again ▼

Explain ↗

Cancel

Clear

Create ▼

☐ Reuse query results
up to 60 minutes ago

Query results

Query stats

✓ Completed

Time in queue: 95 ms

Run time: 798 ms

Data scanned: 2.78 KB

Results (7)

Copy

Download results CSV

Search rows

< 1 > ⚙

#	▼	day	▼	cost	▼
1		2025-11-01		0.0	
2		2025-11-24		-2.00000000000043854E-10	
3		2025-11-25		1.000000150380187E-10	
4		2025-11-26		2.9495968173933853E-21	
5		2025-11-27		1.4000001054281785E-9	
6		2025-11-28		2.000000155072401E-9	
7		2025-11-29		1.4000000924177525E-9	

Query 4 : X

Query 5 : X

Query 6 : X

Query 7 : X

Query 8 : X

> (+) ▼

1 SELECT product_servicecode, SUM(line_item_unblended_cost)

2 FROM finops_cur_db.cur_data

3 GROUP BY product_servicecode;

4

SQL Ln 4, Col 1

Run again

▼

Explain ↗

Cancel

Clear

Create ▼

☐ Reuse query results
up to 60 minutes ago ↗

Query results

Query stats

Query results

Query stats

Completed

Time in queue: 97 ms

Run time: 670 ms

Data scanned: 1.56 KB

Results (15)

Copy

Download results CSV

Search rows

< 1 > ⚙

#	product_servicecode	_col1
1	AmazonSNS	0.0
2	AWSDataTransfer	-8.0000000000006954E-10
3	AWSGlue	0.0
4	AWSEvents	0.0
5	AmazonAthena	-1.3552527156068805E-20
6	AWSSecretsManager	-1.6940658945086007E-21

7	AmazonLocationService	0.0
8	AmazonVPC	2.0990154059319366E-16
9	AmazonCloudWatch	0.0
10	AWSLambda	0.0
11		0.0
12	AmazonS3	-1.9999999868520813E-10
13	awskms	0.0
14	AWSQueueService	0.0
15	AmazonEC2	5.699999837578684E-9

Activity 4.3: Interpret CUR Results

1. Identify expensive services.
2. Check daily spend spikes.
3. Check EC2 idle usage.
4. Note missing EC2 data due to CUR delay.

<
Query 3
✕
✓ Query 4
✕
✓ Query 5
✕
✓ Query 7
✕
✓ Query 8
✕
>
(+)
▼

```

1 SELECT
2     line_item_resource_id AS instance_id,
3     SUM(CAST(line_item_usage_amount AS DOUBLE)) AS total_hours
4 FROM finops_cur_db.cur_data
5 WHERE line_item_product_code = 'AmazonEC2'
6     AND line_item_usage_type LIKE '%BoxUsage%'
7 GROUP BY line_item_resource_id
8 ORDER BY total_hours ASC

```

SQL Ln 8, Col 25

Run again
▼
Explain ↗
Cancel
Clear
Create ▼

☐ Reuse query results
up to 60 minutes ago

1. Run a test invocation.
2. Check CloudWatch logs for execution.

Activity 5.3: Implement Idle Resource Policy

Open AWS Lambda.

● EC2 instance marked as idle if:

- Low usage in CUR (line_item_usage_amount)

- Not used for long periods

- Development workload pattern

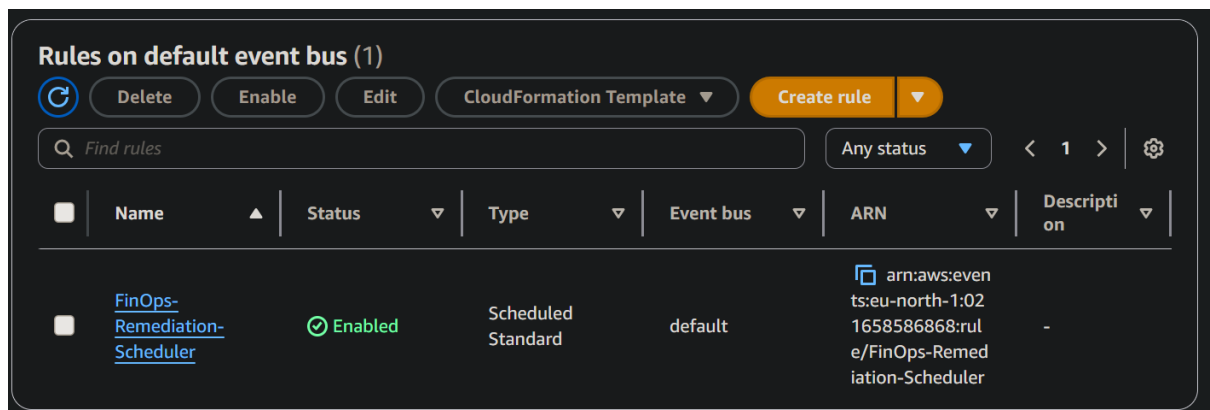
● Lambda applies cleanup actions.

Milestone 6: Automation Scheduling (EventBridge)

This milestone ensures that the FinOps bot runs automatically.

Activity 6.1: Configure EventBridge Rule

1. Open EventBridge → Rules.
2. Create rule:
FinOps-Remediation-Scheduler
3. Use schedule expression (e.g., rate(6 hours)).
4. Set Lambda as target.



Activity 6.2: Validate Automation

1. View next scheduled run.
2. Check CloudWatch for rule invocations.

FinOps-Remediation-Scheduler

Edit
Disable
Delete
CloudFormation Template ▼

Rule details
Info

Rule name FinOps-Remediation-Scheduler	Status Enabled	Event bus name default	Type Scheduled Standard
Description -	Rule ARN arn:aws:events:eu-north-1:021658586868:rule/FinOps-Remediation-Scheduler	Event bus ARN arn:aws:events:eu-north-1:021658586868:event-bus/default	

Event schedule
Targets
Monitoring
Tags

Event schedule
Info

Fixed rate of
6 hour

Edit

Milestone 7: Dashboard & QuickSight Limitation Notice

Activity 7.1: Attempt QuickSight Setup

1. Opened QuickSight signup page.
2. System redirected to **Amazon QuickSuite** instead.
3. QuickSight signup section (username/password) never appeared.
4. QuickSight unavailable for this account/region.

Activity 7.2: Dashboard Decision

- No dashboard created due to QuickSight limitation.
- Excel alternative not used because instructor required QuickSight specifically.
- Athena outputs were used for all analysis.

Milestone 8: Monitoring & Maintenance

Activity 8.1: AWS Monitoring

- Use CloudWatch to monitor:
 - Lambda logs
 - EventBridge triggers
 - Athena query history

- CUR delivery in S3

Activity 8.2: Cost Monitoring

1. Check AWS Billing Dashboard.
2. Verify CUR updates daily.
3. Ensure automation runs without failures.

Conclusion:

The FinOps Automation Engine successfully implements cloud cost governance and remediation techniques using native AWS services including S3, CUR, Glue, Athena, Lambda, and EventBridge. All automation and analysis components worked as expected.

Although Amazon QuickSight was not available in the project's AWS region/account, the rest of the FinOps pipeline was fully implemented, and cost analysis was achieved through Athena SQL queries.

The project demonstrates industry-standard FinOps practices and automates cloud waste reduction effectively.